

Machine Learning: A Comprehensive Textbook

Solutions to Chapter 10 Exercises

Convolutional Neural Networks

Solutions Manual

Exercise 10.1 — Output Dimensions

A $224 \times 224 \times 3$ input passes through: Conv($k = 11, s = 4, p = 0, C_{out} = 96$), MaxPool($k = 3, s = 2$), Conv($k = 5, s = 1, p = 2, C_{out} = 256$). Compute output dimensions and parameter count after each layer. Verify with AlexNet’s architecture.

Solution. The standard output-size formula for a convolution/pooling layer is

$$O = \left\lfloor \frac{I - k + 2p}{s} \right\rfloor + 1,$$

where I is input spatial size, k kernel size, p padding, s stride.

Layer 1: Conv($k = 11, s = 4, p = 0, C_{out} = 96$), input $224 \times 224 \times 3$.

$$O = \left\lfloor \frac{224 - 11 + 0}{4} \right\rfloor + 1 = \lfloor 53.25 \rfloor + 1 = 53 + 1 = 54.$$

Hmm — note AlexNet’s original paper states 55×55 ; this discrepancy arises because AlexNet’s actual input was $227 \times 227 \times 3$ (not $224 \times 224 \times 3$, a well-known erratum in the original paper/many textbooks), which gives $\lfloor (227 - 11)/4 \rfloor + 1 = \lfloor 54 \rfloor + 1 = 55$. Using the problem’s stated input of $224 \times 224 \times 3$ exactly as given: output is **$54 \times 54 \times 96$** .

Parameter count: each output channel has its own $11 \times 11 \times 3$ filter plus a bias: $(11 \times 11 \times 3 + 1) \times 96 = (363 + 1) \times 96 = 364 \times 96 = \mathbf{34,944}$ parameters.

Layer 2: MaxPool($k = 3, s = 2$), input $54 \times 54 \times 96$.

$$O = \left\lfloor \frac{54 - 3}{2} \right\rfloor + 1 = \lfloor 25.5 \rfloor + 1 = 25 + 1 = 26.$$

Output: **$26 \times 26 \times 96$** . *Parameter count:* **0** (pooling has no learnable parameters).

Layer 3: Conv($k = 5, s = 1, p = 2, C_{out} = 256$), input $26 \times 26 \times 96$.

$$O = \left\lfloor \frac{26 - 5 + 2 \cdot 2}{1} \right\rfloor + 1 = \lfloor 25 \rfloor + 1 = 26.$$

Output: **$26 \times 26 \times 256$** (the padding $p = 2$ with a 5×5 kernel and stride 1 is specifically chosen to be “same” padding, preserving spatial size).

Parameter count: $(5 \times 5 \times 96 + 1) \times 256 = (2400 + 1) \times 256 = 2401 \times 256 = \mathbf{614,656}$ parameters.

Verification against AlexNet. Using AlexNet’s actual $227 \times 227 \times 3$ input: Conv1 output is $55 \times 55 \times 96$ (34,944 params, matching), MaxPool1 ($k = 3, s = 2$) gives $\lfloor (55 - 3)/2 \rfloor + 1 = 27$, i.e. $27 \times 27 \times 96$, and Conv2 ($k = 5, s = 1, p = 2, C_{out} = 256$, though AlexNet historically splits this across 2 GPUs/groups) gives $\lfloor (27 - 5 + 4)/1 \rfloor + 1 = 27$, i.e. $27 \times 27 \times 256$ with 614,656

parameters (matching our formula exactly when not accounting for the 2-way grouped-convolution split AlexNet used for GPU memory reasons, which roughly halves the parameter count in the original implementation). The dimension-calculation methodology is confirmed correct; the small numerical difference from the commonly-cited 55×55 is fully explained by the 224 vs. 227 input-size discrepancy noted above.

Exercise 10.2 — Parameter Efficiency

Compare parameters for (a) two 3×3 conv layers on C channels, (b) one 5×5 conv layer on C channels. Generalise: show L stacked 3×3 convs have $\frac{9L}{(2L+1)^2}$ times the parameters of one $(2L+1) \times (2L+1)$ conv.

Solution. Assume all layers map C input channels to C output channels (ignoring bias terms, which are a negligible $O(C)$ additive correction in both cases and don't affect the comparison).

(a) Two stacked 3×3 convs. Each layer has $3 \times 3 \times C \times C = 9C^2$ parameters (kernel spans all C input channels, producing all C output channels). Two such layers: $2 \times 9C^2 = 18C^2$ parameters.

(b) One 5×5 conv. $5 \times 5 \times C \times C = 25C^2$ parameters.

Comparison. $18C^2 < 25C^2$ — two stacked 3×3 convolutions have *fewer* parameters ($18C^2$ vs. $25C^2$, a ratio of $18/25 = 0.72$) than a single 5×5 convolution, *while achieving the same effective receptive field*: two 3×3 convolutions in sequence have a receptive field of 5×5 on the original input (each 3×3 conv expands the receptive field by 2 in each spatial dimension: $1 + 2 + 2 = 5$), matching the 5×5 conv's receptive field exactly. Additionally, stacking two smaller convolutions inserts an extra nonlinearity (activation function) between them, increasing the network's representational capacity/expressiveness beyond what the single larger linear-then-activation layer can achieve — this is precisely the insight behind VGGNet's design philosophy (stacking small 3×3 filters rather than using larger kernels).

General case: L stacked 3×3 convs vs. one $(2L+1) \times (2L+1)$ conv. By the same receptive-field-expansion logic, L stacked 3×3 convolutions have effective receptive field $1 + 2L = 2L + 1$ (each layer adds 2 to the receptive field: after L layers, receptive field = $1 + 2L$), exactly matching a single $(2L+1) \times (2L+1)$ convolution.

Parameter count, L stacked 3×3 convs: $L \times 9C^2 = 9LC^2$.

Parameter count, one $(2L+1) \times (2L+1)$ conv: $(2L+1)^2 C^2$.

Ratio:

$$\frac{\text{stacked } 3 \times 3 \text{ params}}{\text{single } (2L+1) \times (2L+1) \text{ params}} = \frac{9LC^2}{(2L+1)^2 C^2} = \frac{9L}{(2L+1)^2}. \quad \blacksquare$$

For example, at $L = 3$ (matching a 7×7 receptive field, as in VGG's design choice replacing early large filters): ratio = $27/49 \approx 0.551$ — the stacked small-kernel approach uses only about 55% of the parameters of the equivalent single large-kernel convolution, for the same receptive field, while gaining L nonlinearities instead of just 1. As L grows, this ratio behaves as $9L/(4L^2) = 9/(4L) \rightarrow 0$, i.e. the parameter savings from stacking small kernels grow (relatively) ever larger for bigger receptive fields.

Exercise 10.3 — Backprop Through Conv

Derive $\partial L / \partial K_{pq}$ and $\partial L / \partial X_{mn}$ for a 2D convolution layer. Show that the input gradient is a full convolution with a flipped kernel.

Solution. Let the (valid, stride-1) 2D convolution be $Y_{ij} = \sum_{p,q} K_{pq} X_{i+p,j+q}$ (cross-correlation convention, as typically implemented in deep learning frameworks), with kernel K of size $k \times k$ ($p, q = 0, \dots, k-1$).

Gradient w.r.t. the kernel, $\partial L / \partial K_{pq}$. By the chain rule, summing over all output positions that K_{pq} influences:

$$\frac{\partial L}{\partial K_{pq}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \cdot \frac{\partial Y_{ij}}{\partial K_{pq}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \cdot X_{i+p,j+q},$$

since $\partial Y_{ij} / \partial K_{pq} = X_{i+p,j+q}$ directly from the definition of Y_{ij} (only the single term with that specific (p, q) contributes for fixed i, j). Writing $\delta_{ij} := \partial L / \partial Y_{ij}$ (the upstream gradient), this is

$$\frac{\partial L}{\partial K_{pq}} = \sum_{i,j} \delta_{ij} X_{i+p,j+q},$$

which is exactly the (valid) **cross-correlation of the input X with the upstream gradient δ** — i.e. the kernel gradient is computed by convolving (cross-correlating) the original input activations with the backpropagated error signal.

Gradient w.r.t. the input, $\partial L / \partial X_{mn}$. Again by the chain rule, summing over all output positions (i, j) whose receptive field includes input position (m, n) — i.e. all (i, j) such that $m = i+p, n = j+q$ for some valid $p, q \in \{0, \dots, k-1\}$, equivalently $i = m-p, j = n-q$:

$$\frac{\partial L}{\partial X_{mn}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \cdot \frac{\partial Y_{ij}}{\partial X_{mn}} = \sum_{p,q} \delta_{m-p,n-q} K_{pq}.$$

(Here we re-indexed the sum over output positions (i, j) into a sum over kernel offsets (p, q) , using $i = m-p, j = n-q$, and only terms where $(m-p, n-q)$ is a valid output index contribute — implicitly zero-padding δ outside its valid range.)

Now substitute $p' = k-1-p, q' = k-1-q$ (i.e. flip the kernel indices), so $p = k-1-p'$:

$$\frac{\partial L}{\partial X_{mn}} = \sum_{p',q'} \delta_{m-(k-1-p'), n-(k-1-q')} K_{k-1-p', k-1-q'} = \sum_{p',q'} K_{p'q'}^{\text{flip}} \delta_{m-k+1+p', n-k+1+q'},$$

where $K_{p'q'}^{\text{flip}} := K_{k-1-p', k-1-q'}$ is the kernel rotated 180 (flipped along both spatial axes). This last expression is precisely the definition of a **full convolution** of δ with the flipped kernel K^{flip} (a full, rather than valid, convolution — i.e. zero-padded so that every position of the flipped kernel touching any part of δ contributes, extending the output beyond δ 's original spatial extent by $k-1$ on each side). Hence

$$\frac{\partial L}{\partial X} = \delta \circledast K^{\text{flip}} \quad (\text{full convolution with the 180-flipped kernel}). \quad \blacksquare$$

This is exactly why, in a from-scratch convolutional backprop implementation, computing the gradient with respect to the layer's *input* requires convolving the upstream gradient with the kernel rotated by 180 (and using full/zero-padded convolution rather than valid convolution) — a classical and important result in CNN backpropagation, and precisely mirrors how signal-processing textbooks describe the adjoint of a convolution operator.

Exercise 10.4 — Residual Connection Gradient

Show that for a residual block $a^{[l+2]} = F(a^{[l]}) + a^{[l]}$, $\partial L / \partial a^{[l]} = \partial L / \partial a^{[l+2]} \cdot (I + \partial F / \partial a^{[l]})$. Explain why this prevents vanishing gradients.

Solution. By the chain rule, treating $a^{[l+2]}$ as a (vector-valued) function of $a^{[l]}$ via $a^{[l+2]} = F(a^{[l]}) + a^{[l]}$:

$$\frac{\partial L}{\partial a^{[l]}} = \frac{\partial L}{\partial a^{[l+2]}} \cdot \frac{\partial a^{[l+2]}}{\partial a^{[l]}}.$$

Differentiate the residual block's output with respect to its input:

$$\frac{\partial a^{[l+2]}}{\partial a^{[l]}} = \frac{\partial}{\partial a^{[l]}} [F(a^{[l]}) + a^{[l]}] = \frac{\partial F}{\partial a^{[l]}} + I,$$

(the derivative of the identity/skip-connection term $a^{[l]}$ with respect to itself is simply the identity matrix I). Substituting:

$$\frac{\partial L}{\partial a^{[l]}} = \frac{\partial L}{\partial a^{[l+2]}} \cdot \left(I + \frac{\partial F}{\partial a^{[l]}} \right). \quad \blacksquare$$

Why this prevents vanishing gradients. In a plain (non-residual) deep network, the gradient backpropagated through many layers is a *product* of many Jacobian factors $\prod_l \frac{\partial a^{[l+1]}}{\partial a^{[l]}}$, each of which (especially with saturating activations like sigmoid/tanh, Exercise 9.6) tends to have singular values/norm < 1 — so the product of many such factors shrinks *multiplicatively* and can vanish exponentially in network depth.

With residual connections, the local Jacobian at each block is $I + \partial F / \partial a^{[l]}$ rather than just $\partial F / \partial a^{[l]}$ alone — the additive identity term I guarantees that *even if* $\partial F / \partial a^{[l]}$ is very small (e.g. the residual branch has learned to contribute almost nothing, or its Jacobian happens to be near-singular), the gradient can still flow through the network essentially unattenuated via the identity/skip-connection path: $\frac{\partial L}{\partial a^{[l]}} \approx \frac{\partial L}{\partial a^{[l+2]}} \cdot I = \frac{\partial L}{\partial a^{[l+2]}}$ in the extreme case $\partial F / \partial a^{[l]} \rightarrow 0$. Chaining this reasoning across many residual blocks, the total gradient backpropagated to an early layer is (approximately, ignoring second-order cross terms)

$$\frac{\partial L}{\partial a^{[0]}} \approx \frac{\partial L}{\partial a^{[out]}} \cdot \prod_l \left(I + \frac{\partial F_l}{\partial a^{[l]}} \right) = \frac{\partial L}{\partial a^{[out]}} \cdot \left(I + \sum_l \frac{\partial F_l}{\partial a^{[l]}} + \text{higher-order cross terms} \right),$$

which always retains an undiminished additive I term (the “gradient superhighway” provided by the skip connections) regardless of how small or poorly-conditioned any individual residual branch's Jacobian is — in sharp contrast to the plain-network case, where the analogous product has no such guaranteed non-vanishing additive component. This is precisely the mechanism (formalized informally by He et al. 2015/2016) that allows ResNets to be trained successfully with far greater depth (100s of layers) than was practical for plain feedforward or plain-CNN architectures.

Exercise 10.5 — Programming: ResNet Block

Implement a ResNet basic block and bottleneck block in PyTorch. Compare parameters and FLOPs for ResNet-34 (basic blocks) vs. ResNet-50 (bottleneck blocks) for the same accuracy.

Solution. **Reference implementation.**

```

import torch
import torch.nn as nn

class BasicBlock(nn.Module):
    """Used in ResNet-18/34: two 3x3 convs, expansion factor 1."""
    expansion = 1
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)
        self.downsample = None
        if stride != 1 or in_ch != out_ch:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch))

    def forward(self, x):
        identity = x if self.downsample is None else self.downsample(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.relu(out + identity)          # Ex. 10.4:  $F(x) + x$ 

class Bottleneck(nn.Module):
    """Used in ResNet-50/101/152: 1x1 -> 3x3 -> 1x1, expansion factor 4."""
    expansion = 4
    def __init__(self, in_ch, mid_ch, stride=1):
        super().__init__()
        out_ch = mid_ch * self.expansion
        self.conv1 = nn.Conv2d(in_ch, mid_ch, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(mid_ch)
        self.conv2 = nn.Conv2d(mid_ch, mid_ch, 3, stride, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(mid_ch)
        self.conv3 = nn.Conv2d(mid_ch, out_ch, 1, bias=False)
        self.bn3 = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)
        self.downsample = None
        if stride != 1 or in_ch != out_ch:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch))

    def forward(self, x):
        identity = x if self.downsample is None else self.downsample(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.relu(self.bn2(self.conv2(out)))

```

```
out = self.bn3(self.conv3(out))
return self.relu(out + identity)
```

Comparison: ResNet-34 vs. ResNet-50.

Model	Block type	Params	FLOPs (224×224 input)
ResNet-34	Basic (2×3×3)	≈21.8M	≈3.6 GFLOPs
ResNet-50	Bottleneck (1+3+1)	≈25.6M	≈4.1 GFLOPs

Discussion. Despite ResNet-50 being *deeper* (50 vs. 34 layers) it has only slightly more parameters/FLOPs than ResNet-34, and in practice achieves noticeably *higher* top-1 ImageNet accuracy ($\sim 76\%$ vs. $\sim 73.3\%$) — i.e. better accuracy *per FLOP/parameter*. This is precisely because the bottleneck design (1×1 conv to reduce channels, 3×3 conv on the reduced channel count, 1×1 conv to restore/expand channels) drastically reduces the parameter and compute cost of the expensive 3×3 convolution (which now operates on far fewer channels, e.g. reduced by the expansion factor of 4) while still allowing the block’s *output* channel dimension to be large — letting the network go substantially deeper (more blocks, hence more nonlinear compositional depth, cf. Exercise 9.7’s depth-efficiency argument) for a comparable computational budget to a shallower basic-block design. This is exactly why ResNet-50/101/152 (bottleneck-based) became the standard workhorse architectures rather than simply scaling up ResNet-34’s basic-block design to more layers, which would have been considerably more expensive per unit of added depth.

Exercise 10.6 — Transfer Learning Analysis

Why do early CNN layers transfer better than late layers across domains? Propose an experiment to test which layers should be fine-tuned for a medical imaging task using ImageNet-pretrained ResNet.

Solution. Why early layers transfer better. CNNs learn a hierarchical feature representation: early layers learn low-level, generic visual primitives — edges, corners, color blobs, oriented gradients, simple textures — that are essentially universal across virtually all natural-image domains (this has been directly visualized in numerous feature-visualization studies, e.g. Zeiler & Fergus 2014). These low-level statistics (edge orientations, local contrast patterns) are nearly identical whether the images are photographs of dogs, medical X-rays, or satellite imagery, because they reflect basic properties of how light and local structure are captured in any 2D image. Late layers, by contrast, learn increasingly *high-level, task/domain-specific* features — combinations of parts that are specifically useful for distinguishing the 1000 ImageNet object categories (e.g. “dog snout,” “car wheel,” “bird beak” detectors) — which have little semantic relevance to a very different target domain (e.g. detecting tumors in an X-ray, where the relevant high-level structures are anatomically and pathologically specific, not related to everyday object categories at all). Hence early-layer features transfer broadly and robustly, while late-layer features are increasingly specialized to the source task/domain and transfer poorly (or can even actively hurt performance, since they encode source-task-specific biases irrelevant or even misleading for the target task).

Proposed experiment. A systematic layer-freezing/fine-tuning study:

1. Take an ImageNet-pretrained ResNet (e.g. ResNet-50) and, for a medical imaging classification task (e.g. chest X-ray disease classification) with a modest-sized labeled dataset, train several variants that differ *only* in how many layers (counted from the input) are **frozen** (kept at

- pretrained weights) vs. **fine-tuned** (updated on the medical data): e.g. freeze layers 1– k for $k \in \{0, 10, 20, \dots, \text{all}\}$ (fully frozen backbone + new classifier head, all the way up to full end-to-end fine-tuning), replacing/training a new final classification head in every variant.
2. Train each variant with identical hyperparameters (learning rate, epochs, data augmentation, etc.) on the same train/validation split, and evaluate on a held-out medical-imaging test set using an appropriate metric (e.g. AUROC/AUPRC given likely class imbalance, cf. Exercise 7.1).
 3. Plot validation/test performance as a function of k (how many layers were frozen). The expected qualitative pattern, based on the transfer-learning hypothesis above: performance should be relatively insensitive to freezing when k is small (freezing only the earliest, most generic layers costs little, since those features transfer well regardless), but degrade if k is too large (freezing too many layers, including the domain-specific late layers, prevents the network from adapting its high-level feature detectors to the very different visual statistics of medical images) — with an intermediate “sweet spot” k^* typically emerging where freezing the early generic layers (saving compute and reducing overfitting risk, especially valuable with a limited-size medical dataset) while fine-tuning the later, more task-specific layers achieves the best test performance.
 4. As a complementary diagnostic, one could also measure the *magnitude of weight change* during fine-tuning at each layer (e.g. $\|\Delta W_l\|/\|W_l\|$ for a fully-fine-tuned model) — layers that change little are natural candidates for freezing without loss, providing an independent empirical signal (rather than a hypothesis-driven prior) for which layers are domain-specific vs. generic for this particular source-target domain pair.

This kind of systematic ablation directly tests the transfer-learning hypothesis and yields a practical, task-specific recommendation for how many/which layers to fine-tune, rather than relying purely on the general early-vs-late intuition.